

**SIMATS ENGINEERING (SAVEETHA SCHOOL OF
ENGINEERING) Department of Computer Science and Engineering**
ASSESSMENT TOOL 1 – Constraint-Based Problem Solving

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5– Naturalization (BL5,BL6)	Assessment:	Assessment Tool 1 – Constraint Based Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	<i>Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.</i>		
Student Name:	M.S.Dinesh	Reg. No.:	192424016
Section / Batch:	I	Date:	26.08.26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Apply backtracking techniques systematically for combinatorial problem solving.
- Clearly classify problems under P, NP, NP-Hard, and NP-Complete categories wherever required.
- Provide proper justification, state-space representation, and complexity analysis for all solutions.
- Neat presentation and logical explanation are mandatory.

Question Type	Focus Area	Questions
Constraint Based Problem Solving	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP Complete.	Q1

SECTION A – CONSTRAINT-BASED PROBLEM SOLVING

Code of Conduct

I(M.S.Dinesh /192424016) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____ M.S.Dinesh _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%				
Constraint Analysis	25%				
Solution Development	25%				
Application of Concepts	15%				
Justification	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1. Problem Understanding

The Set Partition Problem (or Equal-Sum Partition Problem) is a classical combinatorial decision problem. Given a multiset $S = \{a_1, a_2, \dots, a_n\}$ of positive integers, the objective is to determine whether S can be partitioned into two disjoint subsets, S_1 and S_2 , such that the sum of elements in both subsets is strictly equal.

Mathematical Formulation:

- * Partition Condition: $S_1 \cup S_2 = S$ and $S_1 \cap S_2 = \emptyset$
- * Target Sum Condition: $\sum_{x \in S_1} x = \sum_{y \in S_2} y = \frac{1}{2} \sum_{s \in S} s = \text{Target}$

Input & Output Specification:

- * Input: Array of integers $arr = [1, 5, 11, 5]$
- * Output: Status (Partition Found or No Partition Possible) along with the two partitioned subsets: $S_1 = [1, 5, 5]$ and $S_2 = [11]$.

2. Constraint Analysis

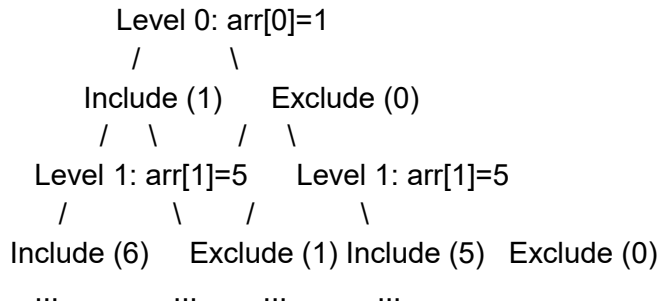
Constraint satisfaction dictates how the combinatorial search space can be bounded and traversed efficiently.

- | Constraint Type | Mathematical Condition | Impact on Search Space |
- | ---|---|---|
- | Parity Constraint | $\text{Total Sum} \bmod 2 == 0$ | If the total sum is odd, an equal integer split is mathematically impossible. The entire search space (2^n states) is pruned immediately at $O(n)$ pre-computation. |
- | Sum Boundary Constraint | $\sum_{x \in \text{subset}} x \leq \text{Target}$ | Any partial subset whose accumulated sum exceeds Target violates feasibility; the entire sub-tree is pruned instantly. |
- | Disjoint Element Constraint | Each element indexed at i is chosen at most once per subset ($x_i \in \{0, 1\}$) | Restricts the state space to a binary decision tree of depth n rather than a general permutation tree. |
- | Target Exactness Constraint | $\text{current_sum} == \text{Target}$ | Terminates exploration upon first successful path discovery, eliminating subsequent searches. |

3. State Space Representation & Pruning Techniques

The search space is structured as a Binary State-Space Tree where each level i (from 0 to $n-1$) represents a decision for element $arr[i]$:

- * Left Branch: Include $arr[i]$ into S_1 ($\text{current_sum} \leftarrow \text{current_sum} + arr[i]$).
- * Right Branch: Exclude $arr[i]$ from S_1 and move to $arr[i+1]$.



Pruning Rules Applied:

- * Odd Total Sum Pruning: If $\sum arr \pmod 2 \neq 0$, terminate immediately without invoking recursion.
- * Over-Capacity Bounding: If $\text{current_sum} > \text{Target}$, backtrack immediately.
- * Early Exit on Success: Return True upward through the recursion stack as soon as a valid partition is found, bypassing all remaining branches.

4. Implementation (Python)

```
def partition_equal_subset(arr):
```

```
    total = sum(arr)
```

```
    # Pruning 1: Odd sum cannot be partitioned into equal halves
```

```
    if total % 2 != 0:
```

```
        return False, [], []
```

```
    target = total // 2
```

```
    n = len(arr)
```

```
    subset1 = []
```

```
    def backtrack(index, current_sum):
```

```
        # Base Case 1: Target achieved
```

```
        if current_sum == target:
```

```
            return True
```

```
        # Base Case 2: Bounding condition / Pruning
```

```
        if current_sum > target or index >= n:
```

```
            return False
```

```
        # Choice 1: Include arr[index]
```

```
        subset1.append(arr[index])
```

```
        if backtrack(index + 1, current_sum + arr[index]):
```

```
            return True
```

```
        # Backtrack step: undo selection
```

```

subset1.pop()

# Choice 2: Exclude arr[index]
if backtrack(index + 1, current_sum):
    return True

return False

if backtrack(0, 0):
    # Derive Subset 2 by removing elements of Subset 1
    subset2 = arr.copy()
    for item in subset1:
        subset2.remove(item)
    return True, subset1, subset2

return False, [], []

# Execution & Testing
if __name__ == "__main__":
    test_input = [1, 5, 11, 5]
    found, s1, s2 = partition_equal_subset(test_input)

    if found:
        print("Partition Found")
        print(f"Subset 1: {s1}")
        print(f"Subset 2: {s2}")
    else:
        print("No Valid Partition Found")

5. Execution Trace (Sample Input: [1, 5, 11, 5])
* Total Sum:  $1 + 5 + 11 + 5 = 22$  (Even  $\rightarrow \text{Target} = 11$ )
* State Trace:
  *  $\text{backtrack}(0, 0) \rightarrow$  Include 1:  $\text{sum} = 1$ 
  *  $\text{backtrack}(1, 1) \rightarrow$  Include 5:  $\text{sum} = 6$ 
  *  $\text{backtrack}(2, 6) \rightarrow$  Include 11:  $\text{sum} = 17 > 11$  (Pruned/Backtrack)
  *  $\text{backtrack}(2, 6) \rightarrow$  Exclude 11
  *  $\text{backtrack}(3, 6) \rightarrow$  Include 5:  $\text{sum} = 11 == \text{Target}$  (Solution Found)
* Result Subsets:
  *  $S_1 = [1, 5, 5]$  (Sum = 11)
  *  $S_2 = [11]$  (Sum = 11)

6. Complexity Analysis & Justification
Time Complexity:

```

* Worst-Case: $O(2^n)$ occurs when no valid subset sums to Target and all paths are explored up to depth n .

* Best-Case: $O(n)$ if the target sum is formed along the first root-to-leaf path or if total sum parity check fails on line 1.

* Auxiliary Space Complexity: $O(n)$ maximum call-stack depth and subset tracking array, avoiding exponential memory allocation.

Complexity Class Categorization:

* Belongs to Class NP: Given a candidate certificate (subset S_1), verification involves computing $\sum S_1$ and checking if it equals $\frac{1}{2} \sum S$. This takes $O(n)$ linear time (deterministic polynomial time).

* Belongs to Class NP-Complete: The Equal-Sum Partition Problem is a direct special case of the Subset Sum Problem (known to be NP-Complete). Any instance of Subset Sum with target T and sum S can be reduced to Equal-Sum Partition in polynomial time by appending an auxiliary element $\frac{1}{2}(S - 2T)$. Because it is both in NP and NP-Hard, it is classified as NP-Complete.

* Completeness Guarantee: The exhaustive backtracking tree explores every possible subset combination systematically until a valid assignment satisfies all constraints, guaranteeing completeness



SIMATS Online Programming Lab
DAA PROGRAMMING

DINESH M
192424016RC

Language: Python C C++ Java

QUESTIONS

Q1

Set Partition Problem (NP-Complete / Backtracking Problem)

100 marks

1/1 answered

Q1 Set Partition Problem (NP-Complete / Backtracking Problem)

100 marks

Given a set of integers, divide it into two subsets with equal sum. Clearly define constraints related to subset formation and sum equality. Analyze how constraints impact the search space. Develop a backtracking solution to generate valid partitions. Apply pruning techniques to eliminate invalid subsets early. Justify how your solution ensures completeness and discuss computational complexity.

Your Code

```
1 def can_partition(arr):
2     total = sum(arr)
3     if total % 2 != 0:
4         return False, []
5
6     target = total // 2
7     n = len(arr)
8     subset = []
9     result = []
10
11     def backtrack(index, current_sum, subset):
12         if current_sum == target:
13             result.append(subset)
14             return True
15         if current_sum > target or index >= n:
16             return False
17
18         subset.append(arr[index])
19         if backtrack(index + 1, current_sum + arr[index], subset):
20             return True
21         subset.pop()
22
23         if backtrack(index + 1, current_sum, subset):
24             return True
```

Input

1 5 11 5

Output

Partition Found
Subset 1: [1, 5, 5]
Subset 2: [11]